

# 编译器设计实验-第五次实验

计算机 2303 尤颢辰

2026 年 5 月 27 日

## 1 实验任务（必做）

### 1.1 任务描述

0. 拓展之前实现的计算 FIRST 集和 FOLLOW 集的函数，使其能考虑空产生式；
1. 设计 AST 节点与语义值数据结构；
2. 为每条产生式绑定语义动作函数（文法扩展）；
3. 在语法分析主循环中维护属性栈，规约时执行对应语义动作；
4. 符号表实现：支持作用域栈结构，处理变量声明和引用；
5. 在语义动作中检测并报告语义错误。

### 1.2 Task0: 使 FIRST 集与 FOLLOW 集计算能考虑空产生式

在实现 Lab4 的时候，考虑不太完整，没有实现  $\varepsilon$  产生式的处理。因此 `compute_first` 只需看右部第一个符号，`compute_follow` 只需看后面紧邻的符号。Lab5 的文法含有空产生式，需要进行修改。

具体来说，计算 FIRST 集的时候要逐符号向右扫描产生式右部，直到扫描到的符号 FIRST 集不包含  $\varepsilon$ 。扫描的过程中把符号的 FIRST 集加入左部的 FIRST 集。计算 FOLLOW 集的时候同样逐符号向右扫描，直到 FIRST 集不含  $\varepsilon$ ，一路上把符号的 FIRST 集加入计算目标的 FOLLOW 集。

### 1.3 Task1: AST 节点与语义值设计

语义分析在 SLR(1) 归约的过程中同步进行，需要两类数据结构：一类用于永久保存分析结果（抽象语法树 AST），一类用于在归约时临时传递属性（语义值）。

1. 设计 AST 节点 `ASTNode` 抽象语法树去掉了具体语法树中的标点符号和中间非终结符，只保留程序结构信息。每个节点包含四个字段：

- `node_type`: 节点种类（用 `ASTNodeType` 枚举表示，如赋值、二元运算、函数声明等）；
- `name`: 附加文本，含义随节点种类而定；
- `val_type`: 该节点的语义类型，供类型检查使用；
- `children`: 子节点列表。

#### 2. 语义值设计

语义值是属性栈的元素，与 SLR(1) 的状态栈对应。每个文法符号在归约时携带自己的综合属性：

- `ast`: 该符号对应的 AST 子树；
- `type`: 表达式符号的语义类型，用于归约时做类型检查；
- `lexeme`: 词素原文，来自 Lab2 的词法分析结果。

移进终结符时，根据 `token` 构造一个叶节点的语义值压栈；归约时弹出右部对应的语义值，组合成新的 AST 子树向上传递。

## 1.4 Task2: 属性栈与语义动作分发

实验四只构建了 SLR(1) 分析表，却从未用它去分析一个真实程序。Task2 实现 `run_semantic`，它在标准 SLR(1) 分析的基础上，额外维护一个与状态栈并排的属性栈，边分析边把语义信息构建成抽象语法树。

它靠两个并排的栈工作：

状态栈：和课本讲的 SLR(1) 分析栈一样，存状态号，决定下一步该干啥；

属性栈：每个符号在这里存一份“语义值”（它对应的 AST 子树、类型）。

每一步看栈顶状态和当前 token，查分析表，做四件事之一：

移进：把 token 当成一个叶子节点放进属性栈；

归约（用第 k 条产生式）：从属性栈取出这条产生式右部的几个符号，交给第 k 条产生式的语义动作去拼成一个新节点，放回栈里；

接受：分析成功，属性栈里剩下的就是整棵 AST 的根；

出错：表里没有对应动作，报语法错误。

其中“归约时交给对应语义动作处理”就是所谓的语义动作分发。Task2 只负责把这个“调用语义动作”的接口完成。

实现之后，我们用代码库的 16.src 做测试，对其进行词法分析后生成 AST 树，结果如下：

```
=== AST ===
Prog
  DeclList
    DeclList
      Decl
        Type (int)
        Id (main)
        ParamList
        DeclList
          DeclList
            Decl
              Type (int)
              Id (d)
            StmtList
              Stmt
                Id (d)
                Num (5) : int
              Stmt
                Id (d)
            Stmt
              Id (main)
              ArgList
```

图 1: AST

## 1.5 Task3: 文法扩展与产生式语义动作实现

Task2 只是把分析跑通，AST 的节点都是非终结符。Task3 要给每条产生式配一段语义动作，让归约建出有意义的抽象语法树节点，并把类型算出来。这样归约，最后栈顶就是一棵带类型标注的抽象语法树。

在这一步一共实现了七种语句（九条产生式），分别为：赋值语句、条件语句、循环语句、返回语句、复合语句块（即用 {} 括起来的语句块）、函数调用语句、输出。还是用 16.src 进行测试，结果如下：

```

=== AST ===
Program
  FunDecl (main) : int
  VarDecl (d) : int
  Assign
    Id (d) : int
    Num (5) : int
  ReturnStmt
    Id (d) : int
  Call (main)

```

图 2: AST(2)

## 1.6 Task4: 符号表实现

语义检查离不开符号表，它记录”哪个名字、什么类型、在哪个作用域”。现在我们实现符号表。

符号表的结构是一个”作用域栈”：栈里每一层是一张”名字 → 符号信息”的表。最底层是全局作用域，每进入一个函数体就压入新的一层。它支持四种操作：进入作用域、退出作用域、声明、查找。

那么符号表在什么时候用？我没有在归约的同时操作符号表，而是分两阶段。规约阶段只负责把 AST 建出来，之后再用一次遍历走这棵 AST，配合符号表做语义分析。这样做的原因是：自底向上解析时，作用域和递归的时机很难处理，而在已经建好的 AST 上遍历，能很自然地进出作用域。

遍历时的规则：遇到函数节点就开一层作用域、登记形参，遇到声明节点就 declare，遇到标识符引用就 lookup，查不到就记一条”未声明”错误。

还是用 16.src 测试，结果如下：

```

=== 符号表 ===
作用域 [全局]:
  main          : int (层级 0)
作用域 [main]:
  d             : int (层级 1)

```

图 3: 符号表

## 1.7 Task5: 语义错误检测

前面 Task4 已经能建符号表，Task5 在同一次 AST 遍历里补上类型检查，让”语义检查”真正起作用。要做类型检查，先得知道每个表达式的类型，所以遍历时自底向上算出每个表达式节点的类型。类型算出来后，做三类检查：重复声明、赋值类型不对、返回值类型不符。加上 Task4 已有的未声明，一共四类语义错误。

下面，我们分别对 16.src 和 17.src 进行语义错误检测，结果如下：

```
=== 语义检查 ===  
无错误
```

图 4: 16.src

```
=== 语义检查 ===  
共 2 条:  
未声明的标识符: c  
未声明的标识符: c
```

图 5: 17.src

可以看到，能正确地检测出语义错误。

## 1.8 输出结果

整个实验中，Lab4 完成的 slr1.cpp 负责解析文法，输出 slr1\_table.txt 文件；Lab2 完成的 Scanner.cpp 负责解析源程序，完成词法分析；Lab5 编写的程序读入分析表和 token 流，做语法 + 语义分析，输出结果。

我们选取 26.src 来进行再一次测试，下面列出其源码与 AST 语法树/符号表的结果：

```
Lab5 > 代码库 > 26.src  
1   int add(int a; int b; ) {  
2   |   return a + b  
3   };  
4  
5   int calc( ) {  
6   |   int sum;  
7   |   sum = add(3, 4,);  
8   |   return sum  
9   };  
10  calc()
```

图 6: 源码

```
PS E:\2026_Junior_S2\CompilerLab> cd e:\2026_Junior_S2\CompilerLab  
>> .\Lab2\scanner.exe "Lab5\代码库\26.src" Lab5\tokens.txt  
>> .\Lab5\semantic.exe Lab5\slr1_table.txt Lab5\tokens.txt  
=== SLR(1) 分析表 (101 状态, 51 产生式) ===  
  
=== Token 流 (42 个) ===  
  
=== AST ===  
Program  
  FunDecl (add) : int  
    Param (a) : int  
    Param (b) : int  
    ReturnStmt  
      BinOp (+) : int  
        Id (a) : int  
        Id (b) : int  
  FunDecl (calc) : int  
    VarDecl (sum) : int  
    Assign  
      Id (sum) : int  
      Call (add) : int  
        Num (3) : int  
        Num (4) : int  
    ReturnStmt  
      Id (sum) : int  
      Call (calc) : int  
  
=== 符号表 ===  
作用域 [全局]:  
  add      : int (层级 0)  
  calc     : int (层级 0)  
作用域 [add]:  
  a        : int (层级 1)  
  b        : int (层级 1)  
作用域 [calc]:  
  sum      : int (层级 1)  
  
=== 语义检查 ===  
无错误
```

图 7: 结果

可以看到，我们成功构建起了 SLR 引导的语义分析框架。

## 2 实验任务（选做）

### 2.1 任务描述

必做部分的词法分析 (Lab2)、SLR(1) 分析表 (Lab4)、语义分析 (Lab5) 都是手工实现的。选做我们使用 flex + bison, 把同一门语言的编译器前端重新搭建一遍, 实现完整流程:

源代码 → token 流 → 抽象语法树 (AST) → 符号表

### 2.2 成果展示

在编写词法规则文件的时候, 关键字写在标识符前、整数和浮点靠最长匹配区分、ID/NUM/FLO 通过 yylval 把词素传给语法分析器。关键代码如下:

```
1 "int"                { return INT; }
2 [a-zA-Z_][a-zA-Z0-9_]* { yylval.str = strdup(yytext); return ID; }
3 [0-9]+\.[0-9]+([eE][+-]?[0-9]+)? { yylval.str = strdup(yytext); return FLO; }
4 [0-9]+               { yylval.str = strdup(yytext); return NUM; }
5 "<="                 { return LE; }
6 "/*".*               { /* 行注释跳过 */ }
```

在编写语法 + 嵌入语义动作的时候, Lab4 的建表、Lab5 的双栈解析都不用写了, 靠 bison 自动生成; 只需描述文法与每条规则建什么节点即可。关键代码如下:

```
1 Expr : Expr ADD Term { $$ = make_binop("+", $1, $3); }
2      | Term          { $$ = $1; }
3      ;
4 Decl : Type ID       { $$ = make_node(AST_VAR_DECL, $2, $1->val_type.c_str()); }
5      | Type ID LPA ParamList RPA LBR DeclList StmtList RBR
6          { $$ = make_node(AST_FUN_DECL, $2, $1->val_type.c_str());
7            splice($$, $4); splice($$, $7); splice($$, $8); }
8      ;
```

编写完成这两个文件之后, 我们还需要复用 Lab5 的语义分析的部分。譬如 AST 结构、符号表、遍历过程等。

接下来我们输入下面的命令构建:

```
1 ..\win_bison.exe -d lab5.y      # 生成语法分析器
2 ..\win_flex.exe  lab5.l         # 生成词法分析器
3 g++ -std=c++17 lab5.tab.c lex.yy.c -o lab5_frontend.exe
```

运行: lab5\_frontend.exe < 源文件.src>

输出: AST, 符号表, 语义检查结果。

就以 16.src 和 17.src 为样例, 看看输出结果:

```

PS E:\2026_Junior_S2\CompilerLab\Lab5\flex_frontend> .\lab5_frontend.exe "..\代码库\16.src"
=== AST ===
Program
  FunDecl (main) : int
  VarDecl (d) : int
  Assign
    Id (d) : int
    Num (5) : int
  ReturnStmt
    Id (d) : int
  Call (main) : int

=== 符号表 ===
作用域 [全局]:
  main : int (层级 0)
作用域 [main]:
  d : int (层级 1)

=== 语义检查 ===
无错误

```

图 8: 16.src

```

PS E:\2026_Junior_S2\CompilerLab\Lab5\flex_frontend> .\lab5_frontend.exe "..\代码库\17.src"
=== AST ===
Program
  FunDecl (add) : int
  VarDecl (a) : int
  VarDecl (b) : int
  Assign
    Id (a) : int
    Num (3) : int
  Assign
    Id (b) : int
    Num (2) : int
  Assign
    Id (c)
    BinOp (+) : int
    BinOp (*) : int
      Id (a) : int
      Id (b) : int
    Num (5) : int
  ReturnStmt
    Id (c)
  Call (add) : int

=== 符号表 ===
作用域 [全局]:
  add : int (层级 0)
作用域 [add]:
  a : int (层级 1)
  b : int (层级 1)

=== 语义检查 ===
共 2 条:
  未声明的标识符: c
  未声明的标识符: c

```

图 9: 17.src

可以看到，自动化工具生成的分析器加上我们自己写的 AST+ 符号表可以正常运行。

### 3 个人总结

本次实验在 Lab2 词法分析、Lab4 SLR(1) 分析表的基础上，完成了语义分析这一编译器前端的最后环节。架构上按照实验要求，让 Lab4 把分析表导出成文件、Lab5 再读入。实验中我遇到了几个问题，其一是文法的风格：PPT 给的两版文法在分号是作分隔符还是终结符上并不一致，官方测试用例遵循主文法的分隔符写法，我把文法重写对齐才让测试程序能被解析。其二是语义分析的组织方式，自底向上解析时，作用域的进出、递归函数名的可见性这些时机很难在归约的同时处理好，于是我先建好 AST，再遍历分析，解决了这个问题。

选做部分，我用 flex + bison 把同一门语言的前端重做了一遍：词法规则交给 flex、文法与归约动作交给 bison 自动生成分析器，而 AST、符号表、类型检查则直接复用我手写的逻辑，最终得到一致的结果。这个过程中，手工实现帮我理解了每一步的原理，而工具则把切词、建表、解析这些机械工作自动化，两者结合才是工程中真正高效的方式。